Project 3

Vulnerability Management Lifecycle

Detection, Remediation, and Verification of CVEs using Wazuh

Author: DAANIYA ABSAR

Environment: Kali Linux Agent (`kali-agent-01`)

Target Package: `weasyprint` (Python Library)

November 7, 2026

Contents

1 Executive Summary **2**

2 Key Concepts & Terminology **2**

3 Methodology Overview **2**

4 Phase 1: Detection and Baseline **3**

5 Phase 2: Remediation Strategy **4**

5.1 Understanding Package Managers 4

5.2 Executing the Upgrade 4

6 Phase 3: Verification and Proof **5**

6.1 Investigating the "Resolved" Event 6

7 Discussion: Interpreting the CVSS -1 Chart **6**

8 Conclusion **7**

1 Executive Summary

This report documents a complete vulnerability-management cycle using Wazuh. Rather than simply listing command outputs, this document explains the *why* behind the actions taken. The project successfully identified four active CVEs associated with the `weasyprint` package on a Kali Linux agent, executed a dual-package-manager remediation strategy, and verified the complete removal of the vulnerabilities.

By understanding the difference between raw vulnerability counts and actual risk, this report demonstrates the core triage skills required in a Security Operations Center (SOC).

2 Key Concepts & Terminology

Before diving into the technical steps, it is crucial to understand the jargon used in vulnerability scanning.

CVE vs. CVSS: What is the difference?

CVE (Common Vulnerabilities and Exposures): Think of this as a "case number." When a flaw is discovered, it is given a CVE ID (e.g., CVE-2026-55073). It simply means, "We know this vulnerability exists."
CVSS (Common Vulnerability Scoring System): Think of this as the "severity ruling." It is a score from 0.0 to 10.0 indicating how dangerous the flaw is.

The Danger of Raw Vulnerability Counts

A dashboard showing "3,230 vulnerabilities" is often misleading. A massive portion of these are unscored (CVSS `-1`) or low-priority. Effective SOC analysts do not panic at the total count; they filter for *Critical* and *High* severities to find what actually requires immediate action.

3 Methodology Overview

The project followed a strict three-phase lifecycle, visualized below:

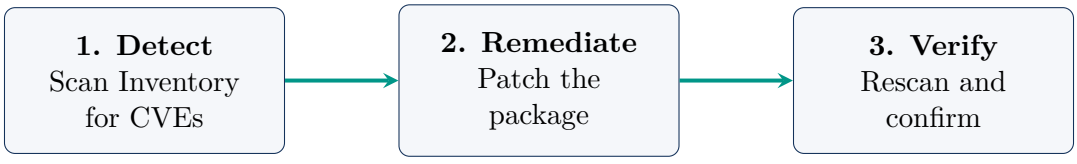


Figure 1: The Detect → Remediate → Verify Lifecycle.

4 Phase 1: Detection and Baseline

The environment initially presented a high volume of vulnerabilities across the board. The overall dashboard showed 105 Critical and 922 High-severity findings.

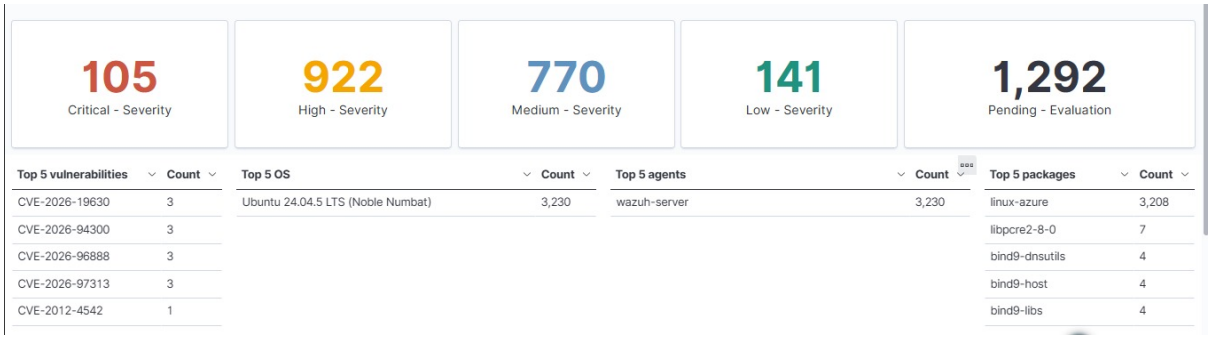


Figure 2: Overall vulnerability dashboard showing severity breakdown.

Using the Wazuh Top 5 dashboards, we narrowed the scope to a specific Kali agent and the Python library `weasyprint`.

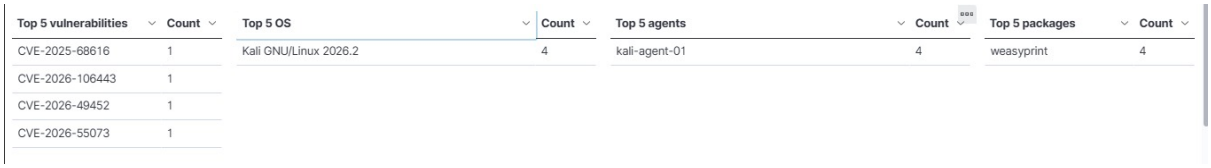


Figure 3: Top 5 agents and packages tables isolating kali-agent-01 and weasyprint.

Filtering the Wazuh Inventory specifically for `kali-agent-01` and `weasyprint` yielded a baseline of exactly **4 active CVEs**.

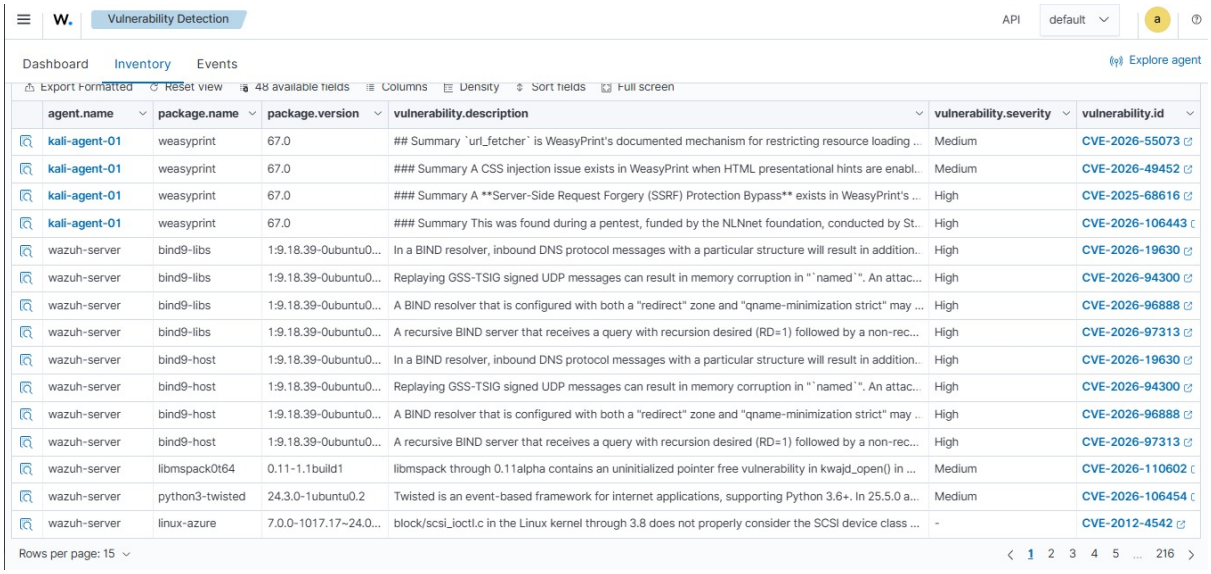


Figure 4: Baseline Inventory: Four active weasyprint CVEs on version 67.0.

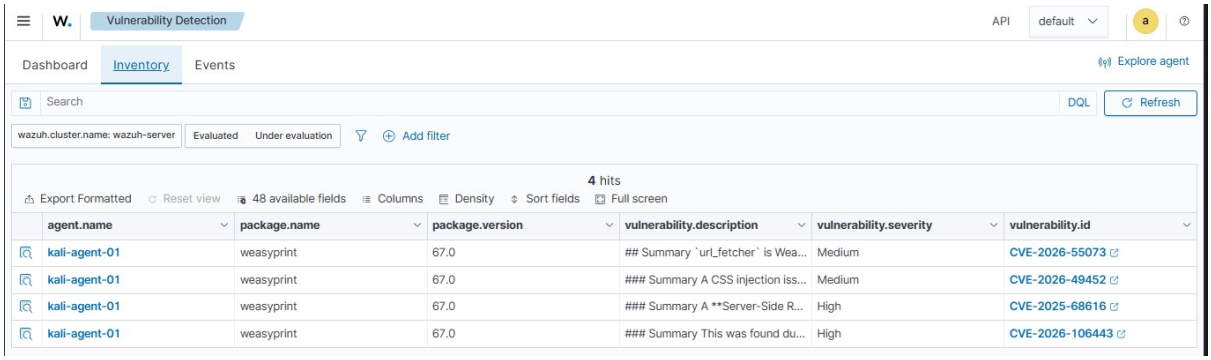


Figure 5: The specific Wazuh DQL query used to isolate the 4 weasyprint CVEs.

5 Phase 2: Remediation Strategy

5.1 Understanding Package Managers

On a Kali Linux system, `weasyprint` can be installed in two ways: via the OS package manager (`apt`) or the Python package manager (`pip`). Often, the OS repository lags behind the official Python library updates. Because the initial version was 67.0, we verified the state via terminal before upgrading.

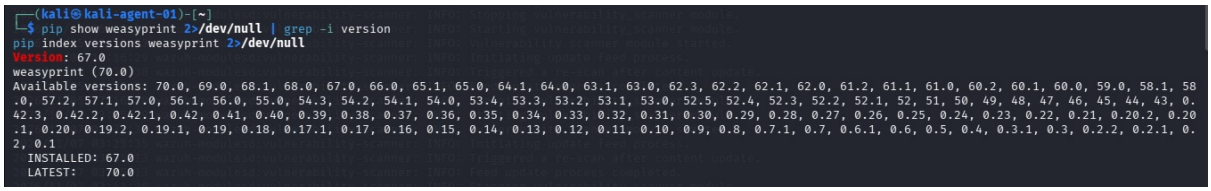


Figure 6: Terminal verification: weasyprint was running version 67.0.

5.2 Executing the Upgrade

To ensure the vulnerability was fully patched, we applied updates via both package managers. First, the standard Debian package:

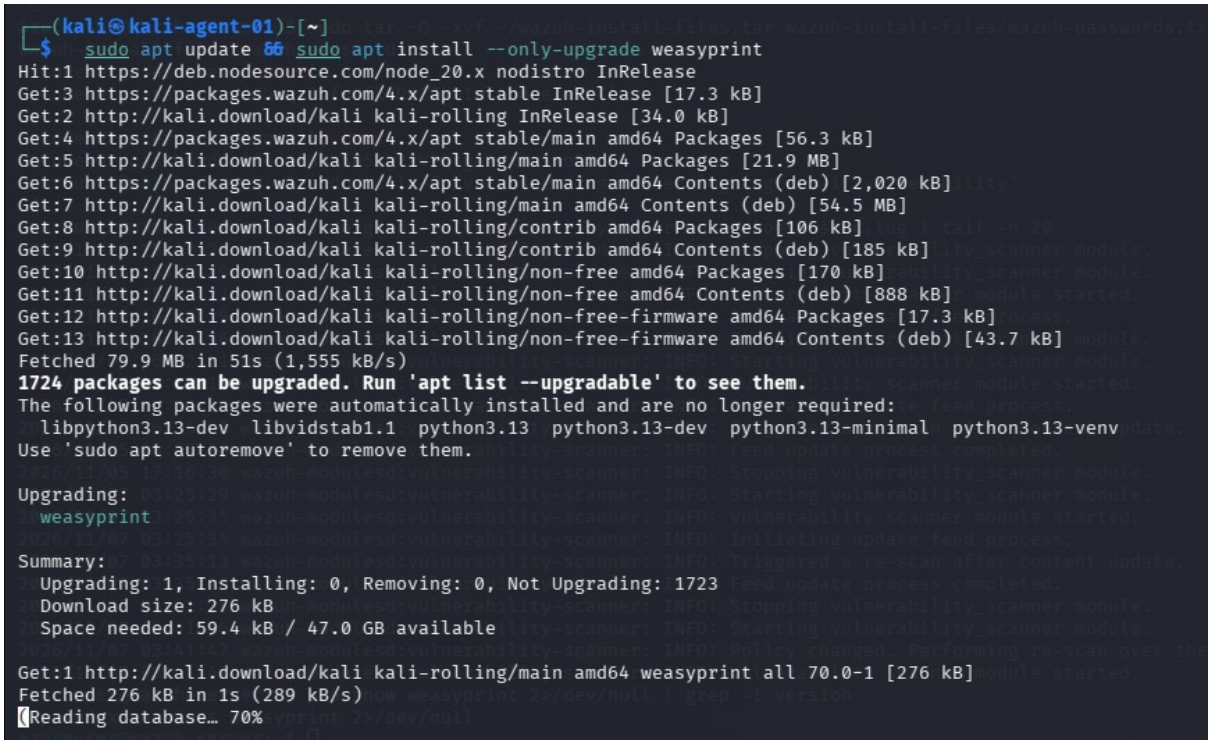


Figure 7: Upgrading weasyprint via the apt package manager.

Next, because the Python module itself needed to be brought up to the latest secure version (70.0), we used `pip` with the `-break-system-packages` flag to override the OS-level restriction and force the update.

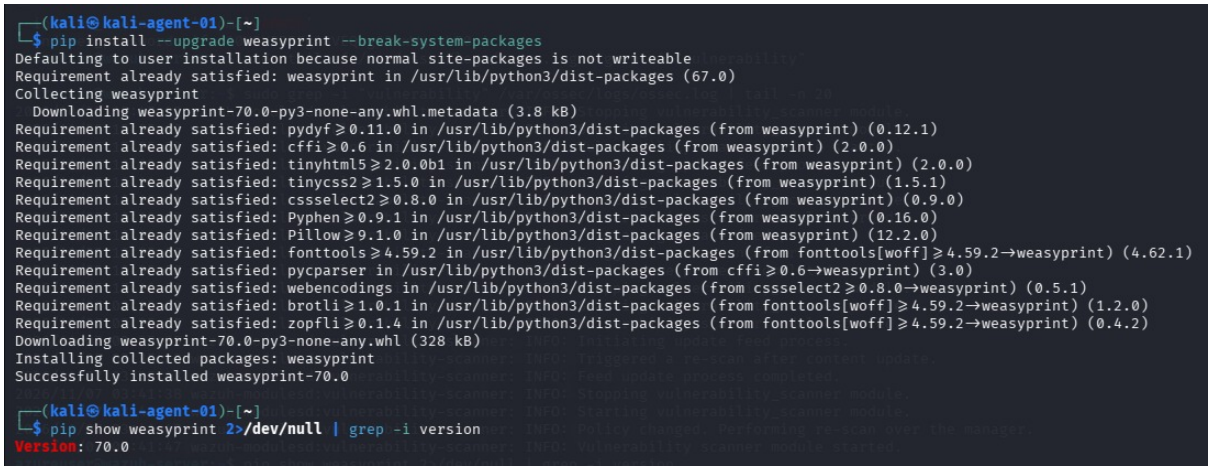


Figure 8: Upgrading weasyprint via pip to version 70.0.

6 Phase 3: Verification and Proof

Following the upgrade, Wazuh was triggered to rescan the agent. We then checked the Wazuh Inventory again. As shown below, the four `weasyprint` CVEs are completely gone.

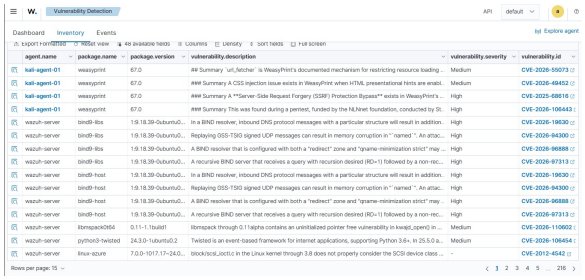


Figure 9: Before: 4 Active CVEs

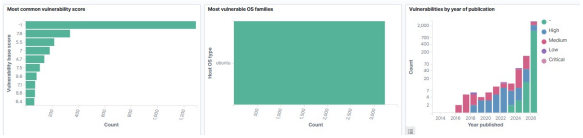


Figure 10: After: 0 Active CVEs

Figure 11: Before-and-after comparison proving successful remediation.

6.1 Investigating the "Resolved" Event

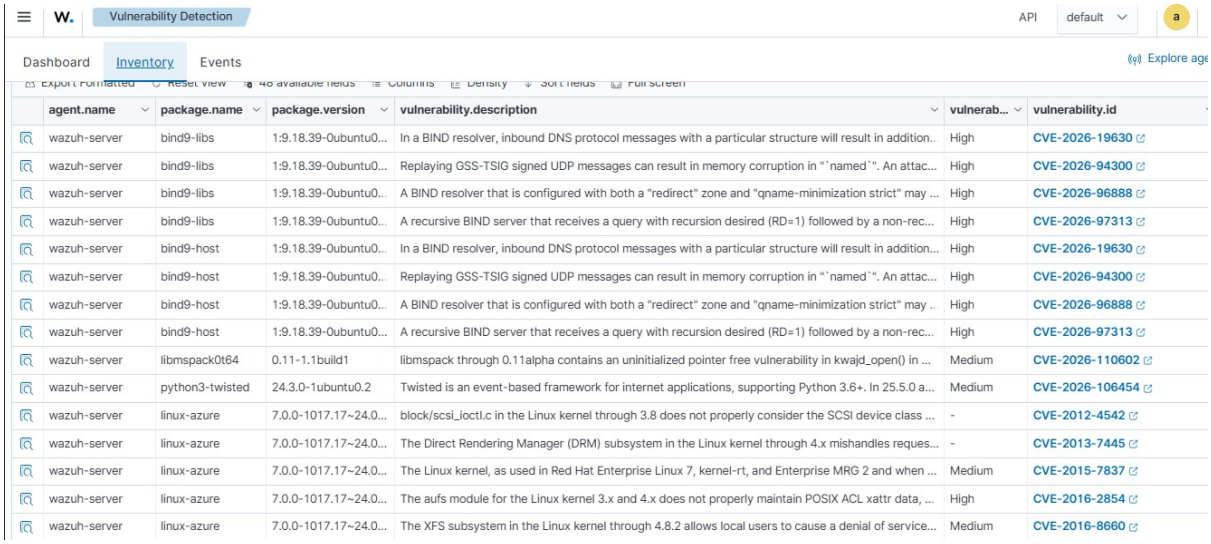
An attempt was made to find a specific "resolved" event in the Wazuh Events log. The search returned no results. **This does not invalidate the remediation.** Wazuh Inventory acts as a live "State" table (showing what is vulnerable *right now*), while the Events log relies on specific rule triggers that may vary by version. The visual inventory comparison above is absolute proof of success.



Figure 12: Events log search returning no explicit "resolved" log, which is expected behavior.

7 Discussion: Interpreting the CVSS -1 Chart

The dashboard initially showed a massive bar at `-1` on the "Most common vulnerability score" chart. This is a common trap for new analysts.



agent.name	package.name	package.version	vulnerability.description	vulnerab...	vulnerability.id
wazuh-server	bind9-libs	1:9.18.39-0ubuntu0...	In a BIND resolver, inbound DNS protocol messages with a particular structure will result in addition..	High	CVE-2026-19630
wazuh-server	bind9-libs	1:9.18.39-0ubuntu0...	Replaying GSS-TSIG signed UDP messages can result in memory corruption in "`named`". An attac...	High	CVE-2026-94300
wazuh-server	bind9-libs	1:9.18.39-0ubuntu0...	A BIND resolver that is configured with both a "redirect" zone and "qname-minimization strict" may ...	High	CVE-2026-96888
wazuh-server	bind9-libs	1:9.18.39-0ubuntu0...	A recursive BIND server that receives a query with recursion desired (RD=1) followed by a non-rec...	High	CVE-2026-97313
wazuh-server	bind9-host	1:9.18.39-0ubuntu0...	In a BIND resolver, inbound DNS protocol messages with a particular structure will result in addition...	High	CVE-2026-19630
wazuh-server	bind9-host	1:9.18.39-0ubuntu0...	Replaying GSS-TSIG signed UDP messages can result in memory corruption in "`named`". An attac...	High	CVE-2026-94300
wazuh-server	bind9-host	1:9.18.39-0ubuntu0...	A BIND resolver that is configured with both a "redirect" zone and "qname-minimization strict" may ..	High	CVE-2026-96888
wazuh-server	bind9-host	1:9.18.39-0ubuntu0...	A recursive BIND server that receives a query with recursion desired (RD=1) followed by a non-rec...	High	CVE-2026-97313
wazuh-server	libmspack0t64	0.11-1.1build1	libmspack through 0.11alpha contains an uninitialized pointer free vulnerability in kwajd_open() in ...	Medium	CVE-2026-110602
wazuh-server	python3-twisted	24.3.0-1ubuntu0.2	Twisted is an event-based framework for internet applications, supporting Python 3.6+. In 25.5.0 a...	Medium	CVE-2026-106454
wazuh-server	linux-azure	7.0.0-1017.17~24.0...	block/scsi_ioctl.c in the Linux kernel through 3.8 does not properly consider the SCSI device class ...	-	CVE-2012-4542
wazuh-server	linux-azure	7.0.0-1017.17~24.0...	The Direct Rendering Manager (DRM) subsystem in the Linux kernel through 4.x mishandles reques...	-	CVE-2013-7445
wazuh-server	linux-azure	7.0.0-1017.17~24.0...	The Linux kernel, as used in Red Hat Enterprise Linux 7, kernel-rt, and Enterprise MRG 2 and when ...	Medium	CVE-2015-7837
wazuh-server	linux-azure	7.0.0-1017.17~24.0...	The aufs module for the Linux kernel 3.x and 4.x does not properly maintain POSIX ACL xattr data, ...	High	CVE-2016-2854
wazuh-server	linux-azure	7.0.0-1017.17~24.0...	The XFS subsystem in the Linux kernel through 4.8.2 allows local users to cause a denial of service...	Medium	CVE-2016-8660

Figure 13: The CVSS score chart. The massive `-1` bar means "Unscored", not "Severe".

The CVSS scale officially runs from 0.0 (Informational) to 10.0 (Critical). A score of `-1` falls outside this scale and simply means the vulnerability has been identified (it has a CVE ID) but the security community has not yet assigned it a severity rating. This usually happens with very new CVEs or legacy kernel bugs. A raw count of 3,230 findings sounds alarming, but once you filter out the unscored `-1` items and focus on the 105 Critical findings, the actual actionable workload becomes much clearer.

8 Conclusion

This project successfully demonstrated the complete detection, remediation, and verification lifecycle. By correctly identifying the dual package management (apt and pip) required for a Python library on Kali, the agent was upgraded to `weasyprint` version 70.0. The before-and-after Wazuh Inventory comparison serves as definitive proof that the threat surface was reduced and the CVEs were eliminated.